

Politechnika Białostocka

Wydział Informatyki

Sprawozdanie projektowe

CourseManager - system zapisów na kursy

Przedmiot: Programowanie aplikacji WWW w języku Java / Systemy Baz Danych

Grupa: PS7

Autorzy:

Michał Szwabowicz

Jakub Wierciszewski

Karol Ziemak

Białystok, 2026

Spis treści

1. Cel i opis projektu
2. Wymagania funkcjonalne
3. Wymagania niefunkcjonalne
4. Role i uprawnienia użytkowników
5. Analiza przyrostu danych
6. Diagram encji i opis relacji
7. Opis tabel, ograniczeń i indeksów
8. Funkcje, procedury, trigger, sekwencje i kursory
9. Środowisko uruchomieniowe i baza danych
10. Podział pracy w zespole
11. Podsumowanie

1. Cel i opis projektu

CourseManager jest aplikacją internetową do zarządzania kursami i zapisami użytkowników na kursy. Projekt został przygotowany w Javie z użyciem Spring Boot. Aplikacja ma część webową z widokami Thymeleaf oraz część REST API. Dane są przechowywane w bazie PostgreSQL uruchamianej lokalnie w kontenerze Docker.

Najważniejszym celem systemu jest umożliwienie przeglądania kursów, zapisywania się na nie, zarządzania kursami i prowadzącymi oraz obsługi ról użytkowników. Projekt zawiera także raporty administracyjne, udostępnianie kursów innym użytkownikom oraz publiczne linki do wybranych kursów.

2. Wymagania funkcjonalne

Wymagania funkcjonalne opisują, co użytkownik może zrobić w aplikacji. Zostały podzielone według głównych obszarów działania systemu.

Obszar	Opis działania
Rejestracja i logowanie	Użytkownik może założyć konto, zalogować się i wylogować z aplikacji. Hasło jest zapisywane w bazie jako hash BCrypt.
Role użytkowników	System obsługuje administratora, użytkownika z pełnymi prawami, użytkownika ograniczonego oraz osobę niezalogowaną.
Kursy	Użytkownik może przeglądać kursy, sprawdzać szczegóły, sortować i filtrować listę. Użytkownik z odpowiednimi prawami może dodawać, edytować i usuwać kursy.
Kategorie	Użytkownik z pełnymi prawami lub administrator może zarządzać kategoriami kursów. Kategoria ma nazwę, opis, status aktywności i datę utworzenia.
Prowadzący	Użytkownik z pełnymi prawami lub administrator może zarządzać prowadzącymi. Prowadzący ma imię, nazwisko, email, specjalizację, biogram i datę utworzenia.
Zapisy na kurs	Zalogowany użytkownik może zapisać się na kurs i anulować swój zapis. System blokuje podwójny aktywny zapis na ten sam kurs.
Limity miejsc	Przy zapisie na kurs sprawdzana jest liczba wolnych miejsc. Jeżeli kurs jest pełny, użytkownik nie zostanie zapisany.
Udostępnianie kursów	Użytkownik z pełnymi prawami lub administrator może udostępnić kurs konkretnemu użytkownikowi albo wygenerować publiczny link.
Panel administratora	Administrator widzi listę użytkowników, może zmieniać role oraz otwierać raporty.
Raporty	Administrator może sprawdzić najpopularniejsze kursy, wolne miejsca oraz uczestników wybranego kursu.
REST API	Aplikacja udostępnia endpointy REST dla kursów, kategorii, prowadzących, zapisów i udostępnień.

3. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne opisują, w jaki sposób aplikacja powinna działać. Nie są one pojedynczą funkcją, ale wpływają na jakość całego systemu.

Wymaganie	Opis
Bezpieczeństwo	Dostęp do części funkcji jest ograniczony rolami. Hasła użytkowników są hashowane przy użyciu BCrypt. Aplikacja nie łączy się z bazą przez konto administratora PostgreSQL.
Poprawność danych	Baza danych posiada ograniczenia NOT NULL, UNIQUE, CHECK oraz klucze obce. Dodatkowo formularze mają walidację po stronie aplikacji.
Czytelność obsługi	Widoki są przygotowane w Thymeleaf i Bootstrap. Formularze mają etykiety, komunikaty błędów i podstawowe poprawki dostępności.
Wydajność	Dla pól często używanych w zapytaniach dodano indeksy. Listy danych mają paginację, aby nie ładować zbyt dużych wyników naraz.
Możliwość uruchomienia	Aplikacja i baza danych są uruchamiane lokalnie przez Docker Compose. Dzięki temu środowisko jest prostsze do odtworzenia.
Rozdzielenie odpowiedzialności	Kod jest podzielony na kontrolery, serwisy, repozytoria, DTO, encje i widoki. Ułatwia to poprawianie i rozwijanie projektu.
Spójność zmian w bazie	Część logiki zapisów na kurs została przeniesiona do procedur i funkcji w bazie danych. Trigger zapisuje zmiany statusu zapisu do tabeli audytu.
Dokumentowanie API	Dla REST API dodano Swagger UI, co ułatwia sprawdzenie dostępnych endpointów.

4. Role i uprawnienia użytkowników

W aplikacji występują role aplikacyjne. Rola decyduje o tym, które widoki i operacje są dostępne dla użytkownika.

Rola	Dostępne możliwości
Niezałogowany	Może wejść na stronę główną, zalogować się, zarejestrować oraz otworzyć publiczny link udostępnionego kursu.
ROLE_LIMITED_USER	Może przeglądać kursy, sprawdzać szczegóły, zapisywać się na kursy i przeglądać własne zapisy.
ROLE_FULL_USER	Ma możliwości użytkownika ograniczonego oraz może zarządzać kursami, kategoriami, prowadzącymi i udostępnieniami.
ROLE_ADMIN	Ma pełny dostęp do systemu, panelu administracyjnego, listy użytkowników, zmiany ról i raportów.

W systemie istnieje konto administracyjne tworzone przy inicjalizacji bazy danych. Login to admin, a hasło to admin. Hasło w bazie danych jest zapisane w formie zahashowanej.

5. Analiza przyrostu danych

Poniższa analiza jest estymacją studencką. Nie opiera się na prawdziwym ruchu produkcyjnym, ponieważ aplikacja jest projektem zaliczeniowym. Założono użycie aplikacji przez jedną grupę lub mały wydział w czasie kilku semestrów.

Założenie	Przyjęta wartość
Nowi użytkownicy w semestrze	około 150
Nowe kursy w semestrze	około 25
Średnia liczba zapisów na jeden kurs	około 20
Udostępnienia kursów w semestrze	około 250
Anulowane zapisy	około 15% wszystkich zapisów
Okres analizy	3 semestry

Tabela	Po 1 semestrze	Po 2 semestrach	Po 3 semestrach	Komentarz
users	150	300	450	Konta studentów i prowadzących używających aplikacji.
courses	25	50	75	Kursy dodawane w każdym semestrze.
enrollments	500	1000	1500	Największy przyrost, ponieważ każdy kurs ma wielu uczestników.
course_shares	250	500	750	Udostępnienia kursów użytkownikom lub przez link.
audit_log	75	150	225	Wpisy audytowe powstające głównie przy anulowaniu zapisów.
categories	10	15	20	Kategorie rosną wolniej, bo są słownikiem pomocniczym.
instructors	20	35	50	Lista prowadzących rośnie wolniej niż liczba kursów.

Największą tabelą w dłuższym czasie będzie tabela enrollments, ponieważ liczba zapisów rośnie szybciej niż liczba kursów. Z tego powodu dodano indeksy na polach user_id, course_id i status. Dzięki temu łatwiej wyszukiwać zapisy danego użytkownika, zapisy dla kursu oraz aktywne zapisy.

6. Diagram encji i opis relacji

Diagram encji został przedstawiony w formie opisowej. Taki zapis jest prosty do odczytania i pokazuje, które tabele są ze sobą połączone.

Encja główna	Relacja	Encja powiązana	Opis
categories	1 : N	courses	Jedna kategoria może mieć wiele kursów. Jeden kurs należy do jednej kategorii.
instructors	1 : N	courses	Jeden prowadzący może prowadzić wiele kursów. Jeden kurs ma jednego prowadzącego.
users	1 : N	enrollments	Jeden użytkownik może mieć wiele zapisów na kursy.
courses	1 : N	enrollments	Jeden kurs może mieć wiele zapisów użytkowników.
users	1 : N	course_shares jako owner_user_id	Jeden użytkownik może utworzyć wiele udostępnień.
users	1 : N	course_shares jako target_user_id	Jeden użytkownik może otrzymać wiele udostępnień. Pole może być puste dla linku publicznego.
courses	1 : N	course_shares	Jeden kurs może zostać udostępniony wiele razy.
enrollments	1 : N logicznie	audit_log	Tabela audytu zapisuje zmiany statusu zapisów. Relacja jest logiczna przez enrollment_id.

Prosty zapis tekstowy modelu danych:

categories 1 --- N courses N --- 1 instructors
users 1 --- N enrollments N --- 1 courses
users 1 --- N course_shares N --- 1 courses
enrollments 1 --- N audit_log (relacja logiczna)

7. Opis tabel, ograniczeń i indeksów

W tej części opisano encje używane w bazie danych. Typy danych i ograniczenia wynikają ze skryptu tworzącego bazę danych w poprawionej wersji v2 aplikacji.

Tabela users

Kolumna	Typ	Ograniczenia	Opis
id	BIGSERIAL	PRIMARY KEY	Identyfikator użytkownika. Tworzy własną sekwencję.
username	VARCHAR(50)	UNIQUE, NOT NULL	Login użytkownika.
password	VARCHAR(255)	NOT NULL	Hash hasła BCrypt.
email	VARCHAR(100)	UNIQUE, NOT NULL	Adres email użytkownika.
role	VARCHAR(30)	NOT NULL, CHECK	Rola: ROLE_ADMIN, ROLE_FULL_USER albo ROLE_LIMITED_USER.

Tabela categories

Kolumna	Typ	Ograniczenia	Opis
id	BIGSERIAL	PRIMARY KEY	Identyfikator kategorii.
name	VARCHAR(50)	UNIQUE, NOT NULL, CHECK długości 3-50	Nazwa kategorii.
description	TEXT	CHECK max 500 znaków	Krótki opis kategorii.
active	BOOLEAN	NOT NULL, DEFAULT TRUE	Informacja, czy kategoria jest aktywna.
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data utworzenia kategorii.

Tabela instructors

Kolumna	Typ	Ograniczenia	Opis
id	BIGSERIAL	PRIMARY KEY	Identyfikator prowadzącego.
first_name	VARCHAR(50)	NOT NULL, CHECK min 2 znaki	Imię prowadzącego.

last_name	VARCHAR(50)	NOT NULL, CHECK min 2 znaki	Nazwisko prowadzącego.
email	VARCHAR(100)	UNIQUE, NOT NULL	Adres email prowadzącego.
specialization	VARCHAR(100)	NOT NULL, CHECK długości 3-100	Specjalizacja prowadzącego.
bio	TEXT	CHECK max 1000 znaków	Krótki biogram.
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data dodania prowadzącego.

Tabela courses

Kolumna	Typ	Ograniczenia	Opis
id	BIGSERIAL	PRIMARY KEY	Identyfikator kursu.
title	VARCHAR(100)	NOT NULL, CHECK min 1 znak	Tytuł kursu.
description	TEXT	CHECK max 2000 znaków	Opis kursu.
start_date	DATE	NOT NULL	Data rozpoczęcia kursu.
max_seats	INT	NOT NULL, CHECK 1-200	Maksymalna liczba miejsc.
category_id	BIGINT	FOREIGN KEY, ON DELETE SET NULL	Powiązanie z tabelą categories.
instructor_id	BIGINT	FOREIGN KEY, ON DELETE SET NULL	Powiązanie z tabelą instructors.

Tabela enrollments

Kolumna	Typ	Ograniczenia	Opis
id	BIGSERIAL	PRIMARY KEY	Identyfikator zapisu.
user_id	BIGINT	FOREIGN KEY, NOT NULL, ON DELETE CASCADE	Użytkownik zapisany na kurs.
course_id	BIGINT	FOREIGN KEY, NOT NULL, ON DELETE CASCADE	Kurs, którego dotyczy zapis.
status	VARCHAR(20)	NOT NULL, DEFAULT AKTYWNY, CHECK	Status: AKTYWNY albo ANULOWANY.
enrollment_date	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data zapisu.

Tabela course_shares

Kolumna	Typ	Ograniczenia	Opis
id	BIGSERIAL	PRIMARY KEY	Identyfikator udostępnienia.
course_id	BIGINT	FOREIGN KEY, NOT NULL, ON DELETE CASCADE	Udostępniany kurs.
owner_user_id	BIGINT	FOREIGN KEY, NOT NULL, ON DELETE CASCADE	Użytkownik tworzący udostępnienie.
target_user_id	BIGINT	FOREIGN KEY, ON DELETE CASCADE	Użytkownik docelowy. Może być puste dla linku publicznego.
token	VARCHAR(80)	UNIQUE, NOT NULL	Token publicznego linku.
active	BOOLEAN	NOT NULL, DEFAULT TRUE	Czy udostępnienie jest aktywne.
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data utworzenia udostępnienia.

Tabela audit_log

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PRIMARY KEY	Identyfikator wpisu audytu.
enrollment_id	BIGINT	NOT NULL	Id zapisu, którego dotyczy zmiana.
old_status	VARCHAR(20)	brak	Poprzedni status zapisu.
new_status	VARCHAR(20)	brak	Nowy status zapisu.
changed_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Data zmiany statusu.

Indeksy

Indeksy przyspieszają wyszukiwanie danych. W projekcie dodano je głównie na kolumnach używanych w filtrach, relacjach i raportach.

Indeks	Tabela i kolumny	Po co jest użyty
uq_enrollments_active_user_course	enrollments(user_id, course_id) WHERE status = AKTYWNY	Blokuje drugi aktywny zapis tego samego użytkownika na ten sam kurs.
idx_enrollments_user	enrollments(user_id)	Szybsze pobieranie zapisów danego użytkownika.
idx_enrollments_course	enrollments(course_id)	Szybsze pobieranie uczestników kursu.
idx_enrollments_status	enrollments(status)	Szybsze filtrowanie aktywnych i anulowanych zapisów.
idx_courses_start_date	courses(start_date)	Używane przy filtrowaniu kursów po dacie.
idx_courses_category	courses(category_id)	Używane przy filtrowaniu kursów po kategorii.
idx_courses_title	courses(title)	Pomaga przy sortowaniu lub wyszukiwaniu po tytule.
idx_users_username	users(username)	Szybsze logowanie i wyszukiwanie użytkownika.
idx_users_email	users(email)	Kontrola unikalności i wyszukiwanie po emailu.
idx_categories_active	categories(active)	Pomaga rozróżniać kategorie aktywne i nieaktywne.
idx_instructors_email	instructors(email)	Kontrola unikalności i wyszukiwanie prowadzącego po emailu.
idx_course_shares_owner	course_shares(owner_user_id)	Lista udostępnień utworzonych przez użytkownika.
idx_course_shares_target	course_shares(target_user_id)	Lista udostępnień otrzymanych przez użytkownika.
idx_course_shares_token	course_shares(token)	Szybkie otwieranie publicznego linku.

8. Funkcje, procedury, trigger, sekwencje i kursory

Część logiki związanej z zapisami na kursy znajduje się w bazie danych. Dzięki temu ważne zasady, takie jak limit miejsc i blokada podwójnych zapisów, są sprawdzane blisko danych.

Nazwa	Typ	Opis działania
liczba_wolnych_miejsc(p_course_id)	funkcja	Oblicza liczbę wolnych miejsc na kursie. Pobiera limit miejsc z tabeli courses i odejmuje liczbę aktywnych zapisów z tabeli enrollments.
lista_uczestnikow_kursu(p_course_id)	funkcja z pętlą/kursorem	Zwraca listę aktywnych uczestników wybranego kursu. W aplikacji jest używana w raporcie uczestników kursu.
zapisz_na_kurs(p_user_id, p_course_id)	procedura	Sprawdza, czy użytkownik nie jest już aktywnie zapisany oraz czy są wolne miejsca. Jeżeli warunki są spełnione, dodaje zapis.
anuluj_zapis(p_enrollment_id)	procedura	Zmienia status zapisu na ANULOWANY. Zapis nie jest usuwany, dzięki czemu zostaje historia.
log_enrollment_status_change()	funkcja triggera	Jeżeli status zapisu się zmienia, funkcja dodaje wpis do tabeli audit_log.
trg_enrollment_status	trigger	Uruchamia się po aktualizacji tabeli enrollments i wywołuje funkcję audytową.

Sekwencje są tworzone automatycznie przez typy BIGSERIAL oraz SERIAL. Dotyczy to między innymi kolumn id w tabelach users, categories, instructors, courses, enrollments, course_shares i audit_log.

9. Środowisko uruchomieniowe i baza danych

Aplikacja jest uruchamiana lokalnie z użyciem Docker Compose. Składa się z dwóch głównych kontenerów: kontenera aplikacji Spring Boot oraz kontenera bazy danych PostgreSQL. Kontenery działają w jednej sieci Dockera, dzięki czemu aplikacja może połączyć się z bazą po nazwie usługi postgresql.

Element	Wartość
Aplikacja	Spring Boot, Java 17 w konfiguracji projektu
Baza danych	PostgreSQL 16
Miejsce działania bazy	lokalny kontener Docker
Nazwa kontenera bazy	coursemanager-postgres
Nazwa kontenera aplikacji	coursemanager-app
Port aplikacji	8081 na komputerze użytkownika -> 8080 w kontenerze
Port bazy danych	5433 na komputerze użytkownika -> 5432 w kontenerze
Nazwa bazy	coursemanager
Sieć Dockera	coursemanager-network
Wolumen danych	postgres_data

Konta bazodanowe

W projekcie rozdzielono użytkowników aplikacji od użytkowników bazodanowych. Aplikacja nie powinna łączyć się przez konto postgres, tylko przez konto z ograniczonymi prawami.

Konto	Hasło	Przeznaczenie
postgres	mysecretpassword	Administrator kontenera PostgreSQL. Nie jest używany przez aplikację do codziennej pracy.
coursemanager_admin	admin_password	Konto administratora bazy danych z pełnymi prawami do struktury i danych.
coursemanager_developer	developer_password	Konto głównie do odczytu danych podczas pracy deweloperskiej.
coursemanager_app	app_password	Konto serwerowe używane przez aplikację do połączenia z bazą.

Kolejność inicjalizacji bazy

Przy pierwszym uruchomieniu kontenera PostgreSQL wykonywane są skrypty SQL podpięte w Docker Compose. Kolejność jest następująca:

1. schema.sql - tworzy tabele, ograniczenia i indeksy.
2. procedures.sql - tworzy funkcje, procedury, trigger oraz tabelę audit_log.
3. init-db-users.sql - tworzy konta bazodanowe i nadaje uprawnienia.
4. data.sql - dodaje dane startowe, między innymi konto administratora, kategorie i prowadzących.

10. Podział pracy w zespole

Podział pracy został przygotowany tak, aby każdy członek zespołu miał podobny zakres odpowiedzialności.

Autor	Zakres pracy
Michał Szwabowicz	Widoki Thymeleaf, formularze dodawania i edycji, walidacja danych, komunikaty błędów, sortowanie i filtrowanie w interfejsie, poprawki dostępności WCAG oraz sprawdzenie działania podstawowego CRUD z poziomu strony.
Jakub Wierciszewski	Projekt bazy danych PostgreSQL, tabele, relacje, constraints, indeksy, funkcje, procedury, trigger audytowy, konta bazodanowe, uprawnienia oraz uruchamianie bazy w Dockerze.
Karol Ziemak	Konfiguracja Spring Security, role użytkowników, panel administratora, REST API, Swagger, obsługa błędów API, raporty administracyjne oraz testowanie przepływów użytkownika po poprawkach.

11. Podsumowanie

CourseManager spełnia główne założenia projektu aplikacji webowej z bazą danych. System posiada kilka encji domenowych, relacje między tabelami, walidację, logowanie, role, operacje CRUD, REST API, raporty oraz elementy logiki zapisów przeniesione do bazy danych. Aplikacja jest uruchamiana lokalnie w Dockerze, co ułatwia odtworzenie środowiska.

Najważniejszymi tabelami w bazie są users, courses, enrollments, categories, instructors oraz course_shares. Dodatkowo tabela audit_log zapisuje zmiany statusu zapisów. Przy większej liczbie danych najbardziej będzie rosła tabela enrollments, dlatego dodano indeksy związane z użytkownikiem, kursem i statusem zapisu.

Projekt jest przykładem aplikacji zaliczeniowej, w której połączono część webową, REST API, bazę PostgreSQL oraz Docker. Zakres projektu jest wystarczający do pokazania działania ról, formularzy, relacji w bazie i podstawowej logiki biznesowej.